

Tema 07 — Paradigma Lógico: la segunda mitad

- Prolog en profundidad
 - Clase 2: unificación, backtracking, corte, aritmética
 - Clase 3: listas, recursión avanzada, meta-predicados, aplicaciones

"Vamos a abrir la caja negra de Prolog."

Bloque
B0 Repaso
B1 Unificación
B2 Resolución SLD
B3 Backtracking
B4 Corte
B5 Negación por falla
B8 Listas

¿Qué traemos de Clase 1?

Concepto	Recordatorio
Hecho	madre(ana, carlos).
Regla	abuelo(X,Z) :- progenitor(X,Y), progenitor(Y,Z).
Consulta	?- abuelo(ana, Z).
Término	átomo, número, variable, estructura
Recursión	ancestro(X,Y) :- progenitor(X,Y). ancestro(X,Y) :- progenitor(X,Z), ancestro(Z,Y).

Preguntas

1. ¿Un hecho Prolog termina en punto?
2. ¿X con mayúscula es variable?
3. ¿padre(juan, X) busca al padre de Juan?
4. ¿Prolog prueba cláusulas en cualquier orden?

Lo que NO vimos todavía

Clase 1 dejó la **caja negra cerrada**:

- ¿Cómo calza X con ana? → Unificación
- ¿Cómo encuentra Prolog la siguiente solución? → Backtracking
- ¿Cómo sumo $2 + 3$? → Aritmética (no es obvia)
- ¿Cómo guardo una lista? → [H|T]
- ¿Cómo mapeo o filtro? → Meta-predicados

¿Qué es "unificar"?

- Unificar dos términos = encontrar una sustitución de variables que los haga sintácticamente idénticos.

No es igualdad. No es comparación. Es un **matching con unificación**:

“Encajar dos piezas de rompecabezas completando las variables en ambos lados.”

La unificación es el **único** mecanismo de pasaje de información en Prolog. No hay = de asignación; hay unificación.

Los 4 casos de unificación

Caso	Ejemplo	Resultado
Átomos iguales	$\text{ana} = \text{ana}$	éxito, sustitución $\{\}$
Variable libre	$X = \text{ana}$	éxito, $\{X/\text{ana}\}$
Dos variables	$X = Y$	éxito, $\{X/Y\}$ (se unen)
Estructura	$\text{padre}(A,B) = \text{padre}(\text{juan},X)$	éxito, $\{A/\text{juan}, B/X\}$

Cuándo falla

```
?- ana = beatriz.  
false.  
  
?- f(X, X) = f(ana, beatriz).  
false.           % X no puede ser ana Y beatriz al mismo tiempo  
  
?- padre(X) = madre(X).  
false.           % funtores distintos  
  
?- f(a, b) = f(a, b, c).  
false.           % aridad distinta (2 ≠ 3)
```


Unificación: el operador = (= / 2)

- Regla mental: = intenta unificar. Si logra, las variables quedan ligadas. Si falla, backtrack.

= **nunca** evalúa expresiones: $?- X = 2 + 3.$ → $X = 2 + 3.$

Unifica la expresión $2+3$, el término, no 5.

Opción el operador "is". Evalúa y unifica.

$?- X \text{ is } 2 + 3.$ → $X = 5.$

```
?- X = 5.  
X = 5.  
  
?- 5 = 5.  
true.  
  
?- X = Y.  
X = Y.           % ambas quedan ligadas mutuamente
```

Algoritmo de unificación (Robinson 1965)

```
unify(t1, t2):  
    if t1 == t2:                                # mismos átomos o variables  
        return ÉXITO  
    if t1 es variable libre:                    # ligo t1 a t2  
        bind(t1, t2); return ÉXITO  
    if t2 es variable libre:  
        bind(t2, t1); return ÉXITO  
    if t1 = f(a1..an) ∧ t2 = f(b1..bn):    # mismo funtor y aridad  
        for i in 1..n: unify(ai, bi)  
        return ÉXITO  
    return FALLA
```

Trazado paso a paso

Unificar $\text{padre}(\text{juan}, \text{hijo}(\text{pedro}, X))$ con $\text{padre}(Y, \text{hijo}(Z, \text{ana}))$.

```
Paso 1: padre(juan, hijo(pedro,X)) ≡ padre(Y, hijo(Z,ana))
        Funtores iguales (padre/2) → unificar argumentos
Paso 2:  juan ≡ Y                      → bind {Y/juan}
Paso 3:  hijo(pedro,X) ≡ hijo(Z,ana)
        Funtores iguales (hijo/2) → unificar argumentos
Paso 4:  pedro ≡ Z                      → bind {Z/pedro}
Paso 5:  X ≡ ana                       → bind {X/ana}
Resultado: {Y/juan, Z/pedro, X/ana}
```

Most General Unifier (Unificador Más General)

Al unificar $f(X)$ con $f(Y)$:

- $\{X/Y\} \leftarrow$ MGU (más general posible)
- $\{X/ana, Y/ana\} \leftarrow$ también unifica pero innecesariamente específica
- Definición: una sustitución σ es más general que τ si existe una sustitución ρ tal que $\tau = \sigma \circ \rho$.
 - $\tau \rightarrow \textit{tau}$
 - $\sigma \rightarrow \textit{sigma}$
 - $\circ \rightarrow \textit{compuesta con}$ (composición de sustituciones)
 - $\rho \rightarrow \textit{rho}$

Prolog siempre computa la **MGU** — no toma decisiones arbitrarias.

Consecuencia práctica: las variables quedan lo más libres posible para el resto de la resolución.

Unificación en uso — hechos

Paso a paso para `madre(ana, X)`:

1. Intenta `madre(ana, X) = madre(ana, carlos)`
→ `{X/carlos}` ✓
2. Usuario pide más `(;)` → backtrack
3. Intenta `madre(ana, X) = madre(ana, beatriz)`
→ `{X/beatriz}` ✓

```
madre(ana, carlos).  
madre(ana, beatriz).
```

```
?- madre(ana, X).  
X = carlos ;  
X = beatriz.
```

Unificación en uso – reglas

1. Unifica con la 1ª regla: $\{X/P, Y/carlos\} \rightarrow$ nuevo goal `madre(P, carlos)`
2. `madre(P, carlos)` unifica con `madre(ana, carlos)` $\rightarrow \{P/ana\}$ ✓
3. Usuario pide más $\rightarrow$ backtrack a la 2ª regla
4. `padre(P, carlos)` busca un hecho `padre(`, carlos) $\rightarrow$ falla/éxito según base

```
progenitor(X,Y) :- madre(X,Y).  
progenitor(X,Y) :- padre(X,Y).  
  
?- progenitor(P, carlos).
```

`=` vs. `==`

- Regla: = transforma (liga variables). == solo inspecciona.

Operador	Qué hace	Ejemplo	Resultado
$X = Y$	Unificación (liga)	?- $X = 5$.	$X = 5$
$X == Y$	Identidad estricta	?- $X == 5$.	false
$X == Y$	(después de ligar)	?- $X = 5, X == 5$.	true
$X \backslash= Y$	No unifica	?- $1 \backslash= 2$.	true
$X \backslash== Y$	No idénticos	?- $X \backslash== 5$.	true

``=..`` (univ): descomponer términos

- Usos:
 - Construir términos dinámicamente
 - Inspeccionar la estructura de un término

```
?- padre(juan, maria) =.. L.  
L = [padre, juan, maria].
```

```
?- T =.. [saludo, hola, mundo].  
T = saludo(hola, mundo).
```

```
?- f(a, b, c) =.. [F | Args].  
F = f, Args = [a, b, c].
```


La estructura más importante

- Listas = el tipo de dato estrella de Prolog.

Representan:

- Sentencias (cadenas de tokens)
- Soluciones múltiples (colecciones)
- Árboles (listas anidadas)
- Restricciones (conjuntos simbólicos)

Prolog sin listas es como Python sin **list**.

Notación

Sintaxis	Significado	
[]	Lista vacía	
[a]	Un elemento	
[a, b, c]	Tres elementos	
`[H \	T]`	Cabeza H, cola T
`[A, B \	T]`	Dos primeros + resto
`[a, b, c \	T]`	Tres primeros + resto

La estructura interna

[1, 2, 3] es azúcar sintáctico de:

O equivalente: [1 | [2 | [3 | []]]].

Es una lista ligada (cons list) — como en Lisp.

- Consecuencia:
 - Acceso a la cabeza: $O(1)$
 - Acceso al n -ésimo: $O(n)$
 - Append: $O(n)$

Pattern matching con listas

```
primero([X|_], X).  
segundo([_, X|_], X).  
ultimos_dos([_|T], A, B) :- T = [A, B].  
  
?- primero([a,b,c], P).          P = a.  
?- segundo([a,b,c], S).          S = b.  
?- ultimos_dos([a,b,c], A, B).  A = b, B = c.
```

Lista vacía vs. no vacía

Patrón base de cualquier recursión sobre listas.

```
esVacia([]).  
noEsVacia([_|_]).  
  
?- esVacia([]).  
true.  
?- esVacia([a]).  
false.  
?- noEsVacia([a]).  
true.  
?- noEsVacia([]).  
false.
```

`length/2`

Built-in en SWI, pero implementarla ayuda a entender la estructura.

```
length([], 0).  
length([_|T], N) :-  
    length(T, N1),  
    N is N1 + 1.  
  
?- length([a,b,c,d],  
N).  
N = 4.
```

`member/2`

- Multi-modo: funciona tanto como test (elemento dado) como generador.

```
member(X, [X|_]).  
member(X, [_|T]) :- member(X, T).
```

```
?- member(2, [1,2,3]).  
true.
```

```
?- member(X, [1,2,3]).  
X = 1 ;  
X = 2 ;  
X = 3 .
```

`append/3` — la joya

- Reversibilidad total: concatena, divide, enumera prefijos.

```
append([], L, L).  
append([H|T], L, [H|R]) :- append(T, L, R).
```

```
?- append([1,2], [3,4], R).  
R = [1,2,3,4].
```

```
?- append(A, B, [1,2,3]).  
A = [], B = [1,2,3] ;  
A = [1], B = [2,3] ;  
A = [1,2], B = [3] ;  
A = [1,2,3], B = [].
```


`append` como generador

```
% ¿Los últimos 2 elementos?  
?- append(_, [X, Y], [a, b, c, d]).  
X = c, Y = d.  
  
% ¿Es sublista?  
subsecuencia(Sub, L) :-  
    append(_, R, L),  
    append(Sub, _, R).  
  
?- subsecuencia([b,c], [a,b,c,d]).  
true.
```

`last/2` y `nth0/3`

```
last([X], X).  
last([_|T], X) :- last(T, X).
```

```
?- last([a,b,c], L).  
L = c.
```

```
?- nth0(0, [a,b,c], X).% X es el elemento en la posición Index (empezando en 0)  
X = a.                  % index 0-based
```

```
?- nth1(2, [a,b,c], X).  
X = b.                  % index 1-based
```

`msort/2` y `sort/2`

Más útiles de lo que parecen — sirven para “conjuntos” pragmáticos.

¿Qué es @>?

Es uno de los operadores:

- @< → menor que (orden estándar)
- @> → mayor que
- @=< → menor o igual
- @>= → mayor o igual

```
?- msort([3,1,2,1,3], L).  
L = [1,1,2,3,3].           % mergesort, conserva  
                          duplicados  
  
?- sort([3,1,2,1,3], L).  
L = [1,2,3].               % sin duplicados, ordenado  
  
?- sort(0, @>, [3,1,2], L).  
L = [3,2,1].               % descendente
```

Mapas / listas de pares

El operador `-/2` es el `Pair` de Prolog. No tiene semántica propia — es solo un término con buen aspecto.

- Uso extendido: `keysort/2` ordena listas de pares por clave.

```
edades([ana-22, beto-30, carla-22,  
dario-45]).
```

```
?- edades(L), member(ana-E, L).  
E = 22.
```

Listas anidadas

Las listas anidadas se recorren con
recursión doble.

```
matriz([[1,2,3],  
        [4,5,6],  
        [7,8,9]]).
```

```
fila(N, M, F) :- nth0(N, M, F).
```

```
?- matriz(M), fila(1, M, F).  
F = [4, 5, 6].
```

`forall/2` y `between/3`

`forall/2` = "para todo X tal que P, Q se cumple". Alternativa limpia a `fail`-driven loop.

```
?- between(1, 5, X), write(X), nl, fail.  
1  
2  
3  
4  
5  
false.  
  
?- forall(between(1,5,X), X > 0).  
true.  
  
?- forall(between(1,5,X), X > 3).  
false.
```

Construyendo listas

Ineficiente por el **append**..
append pega listas al final.

Es ineficiente si se usa en
recursión para construir listas,
porque recorre toda la lista cada
vez.

```
% Lista de los primeros N naturales
rango(1, [1]).
rango(N, L) :-
    N > 1,
    N1 is N - 1,
    rango(N1, L1),
    append(L1, [N], L).

?- rango(5, L).
L = [1,2,3,4,5].
```

Occurs-check: por qué $X = f(X)$ puede fallar

¿Qué devuelve?

- **Prolog ISO:** debería fallar (hay ciclo: $X = f(f(f(\dots)))$)
- **SWI-Prolog por default:** $X = f(X)$. $\leftarrow$ ¡éxito!
(crea término cíclico)

- Riesgo: algoritmos que asumen no-ciclos pueden colgar.

?- $X = f(X)$.

Forzar occurs-check

Decisión de diseño: **occurs-check cuesta tiempo**.
Los implementadores lo desactivan para velocidad.
El estudiante avanzado lo activa cuando desarrolla.



Para la práctica: no te preocupes.
En el **99%** de los programas Prolog que vas a escribir este año, occurs-check **no es un problema**.
SWI lo desactiva por defecto por una razón: es lentísimo y raramente importa. Lo veremos de nuevo solo si trabajás con estructuras que puedan generar ciclos. **Seguimos.**

```
?- unify_with_occurs_check(X, f(X)).  
false.  
  
% 0 activar globalmente:  
?- set_prolog_flag(occurs_check, true).  
true.  
  
?- X = f(X).  
false.
```

Variable anónima `\_`

```
?- madre(ana, _).           % "¿Ana es madre de alguien?" – sí/no
true.

?- padre(_, _).            % "¿Hay algún padre?"
true.

?- _ = _.                  % dos variables anónimas distintas
true.                      % se unifican (ambas libres)
```

pattern matching

- Pattern matching: hacer coincidir un dato con una forma y extraer sus partes.
- Prolog hace pattern matching de forma nativa: la consulta se compara con una estructura y, si coincide, las variables quedan ligadas.
- Prolog lo hace con unificación
Python y TypeScript lo hacen con desestructuración (assignment)

```
primerElemento([X|_], X).  
tercerElemento([_,_,X|_], X).  
esPar(par(_, _)).
```

```
?- primerElemento([1,2,3], E).  
E = 1.
```

```
?- esPar(par(ana, beto)).  
true.
```

Profundidad infinita (trampa)

Esto suele ser un bug. Revisar el occurs-check o usar `write_canonical/1` para ver la estructura.

```
lista_rara([a | X]).
```

```
?- lista_rara([a | X]).  
X = X. % X permanece  
libre – es una lista  
"abierta"
```

```
?- X = [1,2,3 | X]. %  
¡término cíclico sin  
occurs-check!  
X = [1, 2, 3, 1, 2, 3, 1, 2,  
3|...].
```

Ejercicio relámpago

¿Qué imprime cada consulta?

- (Respuestas en la siguiente filmina.)

```
?- X = 3, Y = 4, X + Y == 7.  
?- X = 3, Y = 4, X + Y == 7.  
?- f(a, Y) = f(X, b).
```

Respuestas

- Lección: `==`, `=`, `:=` son tres operadores distintos. No confundirlos.

```
?- X = 3, Y = 4, X + Y == 7.  
false.           % 3+4 es el término "+(3,4)", distinto  
del átomo 7
```

```
?- X = 3, Y = 4, X + Y := 7.  
true.           % := evalúa aritméticamente
```

```
?- f(a, Y) = f(X, b).  
X = a, Y = b.    % unificación estructural clásica
```

¿Qué es SLD?

- SLD = Selective Linear Definite clause resolution

Algoritmo de inferencia que usa Prolog:

- **Selective**: hay una regla para elegir qué goal probar (el de más a la izquierda)
- **Linear**: cada paso consume un goal y produce una resolvente
- **Definite**: solo cláusulas definidas de Horn (una cabeza, cuerpo conjuntivo)

Resolución = *refutación* de la negación del objetivo.

Vocabulario

Término	Significado
Goal	Objetivo a demostrar
Resolvente	Conjunción de goals pendientes
Cláusula	Hecho o regla del programa
Sustitución θ	Mapeo variable $\rightarrow$ término
Refutación	Demostración por contradicción
Prueba	Cadena de sustituciones que vacía la resolvente

El algoritmo SLD (paso a paso)

```
PROGRAM P, GOAL G:
  resolvente := [G]
  sustituciones := {}
  mientras resolvente no vacía:
    elegir primer goal de resolvente: A
    elegir cláusula de P (top-down): H :- B1, ..., Bn
    si unify(A, H) con MGU  $\theta$ :
      resolvente := (B1, ..., Bn, rest(resolvente))  $\theta$ 
      sustituciones := sustituciones  $\circ \theta$ 
    si no unifica ninguna  $\rightarrow$  BACKTRACK
  retornar sustituciones
```

Ejemplo integrado

Base:

Consulta: `?- abuelo(ana, N).`

```
madre(ana, carlos).  
padre(carlos, laura).  
progenitor(X,Y) :- madre(X,Y).  
progenitor(X,Y) :- padre(X,Y).  
abuelo(X,Z) :- progenitor(X,Y), progenitor(Y,Z).
```

Árbol SLD – nodo raíz

```
abuelo(ana, N)
  |
(regla abuelo = {X/ana, Z/N})
  |
  v
progenitor(ana, Y), progenitor(Y, N)
```

Eficiencia y backtracking

Backtracking exhaustivo = **explosión combinatoria**.

Con 10 goals binarios independientes: $2^{10} = 1024$ ramas.

- Estrategias:
- Poner los goals más restrictivos primero (filtrar temprano)
- Ordenar cláusulas para que la probable primero
- Usar ! (corte) para podar

"Programar bien en Prolog = diseñar el árbol de búsqueda."

Regla de selección

Prolog siempre selecciona el **goal más a la izquierda** (leftmost).

Consecuencia: en

primero resuelve **progenitor(X, Y)**, después **progenitor(Y, Z)**.

¿Por qué importa?

- El orden cambia la eficiencia (no la corrección, en teoría)
- En la práctica, el mal orden puede caer en loops infinitos

```
abuelo(X,Z) :- progenitor(X,Y), progenitor(Y,Z).
```

Orden malo = loop infinito

Prolog expande **ancestro(X,Z)**
indefinidamente antes de tocar el caso
base.

- Regla de oro: en recursiones, caso base primero, o asegurar terminación con una condición de corte.

```
% Versión MALA de ancestro:  
ancestro(X, Y) :- ancestro(X, Z), progenitor(Z, Y).    % recursión primero  
ancestro(X, Y) :- progenitor(X, Y).  
  
?- ancestro(ana, laura).  
% 🔥 BUCLE INFINITO 🔥
```

Regla de escritura

La **regla de escritura** (computation rule) = qué cláusula probar.

Prolog usa **top-down**: de la primera cláusula hacia abajo.

Consecuencia: **el orden de las cláusulas importa** para:

- Eficiencia (poner caso común primero)
- Terminación (caso base antes del recursivo)
- Comportamiento del **!**.

Prueba = demostración

Una **respuesta** de Prolog NO es "encontré un valor".

Es: "**He construido una demostración formal de que tu goal es verdadero, usando estas cláusulas y estas sustituciones.**"

Eso lo hace único entre lenguajes: cada ejecución es un teorema demostrado.

"Logic program execution is theorem proving." — Kowalski, 1974

Retorno con `;`

Cada **;** fuerza a Prolog a **rehacer la última elección** y buscar una demostración alternativa.

. → acepta la solución actual y detiene la búsqueda.

```
?- progenitor(ana, Y).  
Y = carlos ;           % primera demo  
Y = beatriz ;          % segunda demo (backtrack)  
false.                 % no hay más
```

¿Y si no hay ninguna demostración?

Prolog **no dice "no existe"** — dice "no pude demostrarlo con esta base". (negación por falla).

Es la famosa **Closed World Assumption**: lo que no está probado, no está en la base.

```
?- abuelo(tomas, X).  
false.
```

¿Qué es el backtracking?

Cuando una rama del árbol SLD **falla**, Prolog **retrocede** hasta el último **punto de elección** y prueba la siguiente alternativa.

Es **automático**: no lo programás, viene con el motor.

"Backtracking is the engine's memory of 'roads not taken'."

Punto de elección

Se crea un **punto de elección** (*choice point*) cuando:

1. Hay **más de una cláusula** que unifica con un goal, o
2. Un goal tiene múltiples respuestas (hechos con mismo funtor)

El motor lo apila en el **stack de elecciones** para poder volver si la rama falla.

Comparación: como un `try { } catch { ...try again... }` con backup completo del estado.

Ejemplo canónico

```
color(rojo).  
color(verde).  
color(azul).  
  
?- color(X).  
X = rojo ;           % 1er choice point  
X = verde ;         % 2do choice point  
X = azul.           % última opción, no deja choice point  
false.              % ya no hay más
```

`fail`-driven loop

Patrón clásico Prolog para "iterar por todas las soluciones":

El **fail** fuerza backtracking tras cada **write**. Así se imprimen **todas** las soluciones sin recolectarlas.

En 2026 preferimos **forall/2** o **findall/3**, pero **fail** sigue siendo pedagógicamente útil.

```
?- color(X), write(X), nl, fail.  
rojo  
verde  
azul  
false.
```

Árbol de búsqueda con bifurcación

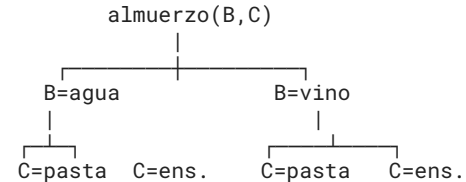
Base:

Consulta ?- almuerzo(B, C) .:

- 4 soluciones – DFS las enumera en orden.
- DFS = Depth-First Search (búsqueda en profundidad)

```
bebida(agua).  
bebida(vino).  
comida(pasta).  
comida(ensalada).
```

```
almuerzo(B, C) :- bebida(B), comida(C).
```



Restricción con `\=`

Cuando `B = C`, falla y backtrackea. Cuando son distintos, sigue.

```
almuerzo_raro(B, C) :-  
    bebida(B), comida(C),  
    B \= C.           % solo se cumple si son distintos  
  
?- almuerzo_raro(agua, X).  
X = pasta ;  
X = ensalada.
```


Anatomía del stack de backtracking

Prolog mantiene **dos stacks**:

1. Stack de control → llamadas/retornos (como cualquier lenguaje)
2. Trail → log de ligaduras de variables

Al fallar:

- El stack de control vuelve al último choice point
- El **trail deshace las ligaduras** hechas desde ese punto → las variables vuelven a estar libres

Esta *restauración destructiva* es lo que Hace Prolog "mágico".

Múltiples soluciones con reglas

```
par(X,Y) :- color(X), color(Y), X \= Y.
```

```
?- par(rojo, Y).
```

```
Y = verde ;
```

```
Y = azul.
```

```
?- par(X, Y).
```

```
% 6 soluciones (3 x 2 posibles)
```

```
X = rojo, Y = verde ; X = rojo, Y = azul ;
```

```
X = verde, Y = rojo ; X = verde, Y = azul ;
```

```
X = azul, Y = rojo ; X = azul, Y = verde.
```

¿Cuándo NO hace backtracking?

Menos choice points = ejecución más rápida. El compilador Prolog detecta "determinismo" y optimiza (WAM).

```
% Con una sola cláusula que unifica → no hay choice point
unico([X]) :- atom(X).

?- unico([hola]).
true.                % no deja "resto" de exploración
```

Árbol de backtracking — ejemplo no trivial

Consulta ?- **conoce(ana, Q)** . produce:

- beto, carla (amigos directos)
- dario, elena (amigos de beto)
- fran (amiga de carla)

```
amigo(ana, beto).  
amigo(ana, carla).  
amigo(beto, dario).  
amigo(beto, elena).  
amigo(carla, fran).
```

```
conoce(X, Y) :- amigo(X, Y).  
conoce(X, Z) :- amigo(X, Y), conoce(Y, Z).
```

Infinito en backtracking

Prevención:

- Llevar lista de visitados.
- Usar **tabling** (extensión SWI).

```
% Si hay ciclos en la base:  
amigo(ana, beto).  
amigo(beto, ana).          % ← ciclo
```

```
conoce(X, Z) :- amigo(X, Y),  
               conoce(Y, Z).
```

```
?- conoce(ana, Q).  
% Puede entrar en bucle infinito:  
ana→beto→ana→beto...
```

El costo oculto del backtracking

- Tiempo: $O(\text{productos cartesianos} \times \text{unificaciones})$
- Espacio: crece con el número de choice points vivos
- Memoria: el trail puede ocupar MB en consultas pesadas

En producción:

- Índices en el primer argumento (automático en SWI)
- `nth_clause/3, jiti_list/1` para observar qué se indexa
- Reordenar para fallar temprano

Backtracking vs. streams

Son duales: un generador Python $\approx$ un predicado Prolog con múltiples respuestas.

	Backtracking Prolog	Generadores/Streams
Control	Automático	Explícito (yield)
Memoria	Trail	Frame de iterador
Reuso	Recomputación	Memoización posible
Paralelismo	No trivial	async/await
Ejemplo	?- member(X, L).	for x in L

Cheatsheet de backtracking

Truco	Sintaxis
Forzar siguiente solución	; en REPL
Parar en la primera	once(Goal)
Colectar todas	findall(T, G, L)
Ignorar fracaso	catch(G, _, true)
Backtrack forzado	fail
Sin backtracking	! (siguiente bloque)

El operador `!`

! (*cut*) = compromiso irrevocable con la rama actual.

Efecto:

1. Quita los choice points **a la izquierda** del **!** en la cláusula actual
2. Quita el choice point de la propia cláusula (no se probarán más cláusulas del mismo predicado)

¿Por qué cortar?

Tres razones:

1. Eficiencia: evitar recomputación de ramas que sabemos que fallarían
2. Determinismo: garantizar que un predicado dé una sola respuesta
3. Expresar "else": codificar alternativas exclusivas

`max/3` — caso canónico

Sin corte (correcto pero con choice point de sobra):

Con corte:

Ambos devuelven el mismo resultado — el
! solo elimina un choice point inútil.

```
max(X, Y, X) :- X >= Y.  
max(X, Y, Y) :- X < Y.
```

```
max(X, Y, X) :- X >= Y, !.  
max(_, Y, Y).
```

Corte verde vs. rojo

- Preferencia ética: solo corte verde. El rojo es para cuando no hay alternativa.

Tipo	Efecto	Si lo quito...
Verde	Solo eficiencia	El programa sigue funcionando igual
Rojo	Cambia la lógica	El programa responde distinto o mal

Ejemplo de corte rojo

- Sin el !:

El **!** es *esencial* aquí — sin él, la semántica rompe.

```
abs(X, X) :- X >= 0, !.  
abs(X, Y) :- Y is -X.
```

```
?- abs(3, Z).  
Z = 3. % correcto
```

```
?- abs(-5, Z).  
Z = 5. % correcto
```

```
?- abs(3, Z).  
Z = 3 ; % correcto  
Z = -3. % ¡MAL! también se cumple "Y is -X"
```

`if-then-else` explícito

SWI-Prolog soporta sintaxis explícita:

Equivalente a la versión con **!**, pero más **legible** y más **local** (no afecta a otras cláusulas).

- Guía en 2026: preferir `(-> ;)` sobre `!` siempre que se pueda.

```
abs(X, Y) :-  
    ( X >= 0  
    -> Y = X  
    ; Y is -X  
    ).
```

Corte, `!` y negación

Implementación clásica de negación por falla:

Lectura:

- Si **P** es demostrable → **!** y **fail** → **not(P)** falla
- Si **P** falla → se prueba segunda cláusula → **not(P)** es verdadero

Así se construye **\+/1** en sistemas mínimos.

```
not(P) :- call(P), !, fail.  
not(_).
```

Corte rojo: el contraejemplo ejecutable

- Mismo predicado — con y sin corte.
Cambia el resultado.

Los cortes eran **rojos**: filtraban resultados. Sin ellos, se rompe la semántica.

- Reescritura declarativa pura (sin corte):

Ahora las condiciones son **mutuamente excluyentes** → no hace falta corte.

- Moraleja: si usás corte preguntá si se puede reformular con condiciones mutuamente excluyentes.

```
% CON cortes
clasificar(X, positivo) :- X > 0, !.
clasificar(X, negativo) :- X < 0, !.
clasificar(_, cero).
```

```
?- clasificar(5, C).
C = positivo.
```

% ← una sola respuesta ✓

```
% SIN cortes
clasificar(X, positivo) :- X > 0.
clasificar(X, negativo) :- X < 0.
clasificar(_, cero).
```

```
?- clasificar(5, C).
C = positivo ;
C = cero.
```

% ← ¡dos! la segunda es errónea ✗

```
clasificar(X, positivo) :- X > 0.
clasificar(X, negativo) :- X < 0.
clasificar(X, cero) :- X =:= 0.
```


Reglas de uso responsable del corte

Regla	Justificación
Preferir (-> ;)	Más local y legible
Sólo corte cuando se pueda	Preserva declaratividad
Documentar corte rojo con comentario	Debugging futuro
No usar ! en predicados de librería	Puede romper invariantes
Probar sin ! primero	Detectar falta de casos